

Como o Transformer funciona

Uma explicação em alto nível, com um modelo treinado do zero

Resumo

O Transformer, apresentado em *Attention Is All You Need* (Vaswani et al., 2017), substituiu as redes recorrentes em tradução automática e virou a base de praticamente todos os modelos de linguagem atuais. Este relatório explica como ele funciona, do texto de entrada à frase traduzida, sem entrar no código. Todas as figuras de resultado vêm de uma implementação do zero em PyTorch, treinada no conjunto Multi30k (alemão→inglês) na configuração *base* do artigo: $N = 6$ camadas, $d_{\text{model}} = 512$, $h = 8$ cabeças, $d_{\text{ff}} = 2048$, 49,1 milhões de parâmetros. Os números mostrados são desse modelo, não ilustrações.

1 O problema que ele resolve

Traduzir uma frase exige olhar para ela inteira. Em “Ein Mann geht”, para escolher o verbo em inglês é preciso saber quem é o sujeito. Antes de 2017 isso era feito com redes recorrentes, que leem a frase palavra por palavra e carregam um estado de memória. Duas consequências ruins: a leitura é **sequencial**, então não dá para paralelizar o treino; e a informação entre duas palavras distantes precisa atravessar todos os passos intermediários, o que a degrada.

A proposta do Transformer é abandonar a recorrência e deixar cada posição olhar diretamente para todas as outras, de uma vez. O caminho entre duas palavras quaisquer passa a ter comprimento 1, e a frase inteira é processada em paralelo. O mecanismo que faz isso é a **atenção**, e é a única peça realmente nova da arquitetura.

2 A arquitetura em uma figura

O modelo tem duas metades (Figura 1). O **encoder** lê a frase de origem e produz uma representação dela; o **decoder** gera a tradução, um pedaço por vez, consultando essa representação. Cada metade é uma pilha de $N = 6$ camadas idênticas em estrutura, mas com pesos independentes.

Vale reparar em três coisas nessa figura, porque elas organizam todo o resto:

- A entrada de cada metade é a soma de um *embedding* com uma *codificação posicional*. Sem essa soma o modelo não teria noção de ordem, já que a atenção trata a frase como um conjunto.
- Cada bloco cinza tem sempre o mesmo formato: um sublayer (atenção ou rede feed-forward), seguido de *Add & Norm*. Isso se repete $2N$ vezes no encoder e $3N$ no decoder.
- O decoder tem um sublayer a mais que o encoder: a *cross-attention*, a seta que sai do encoder e entra no meio do decoder. É por ali que a tradução consulta a frase original.

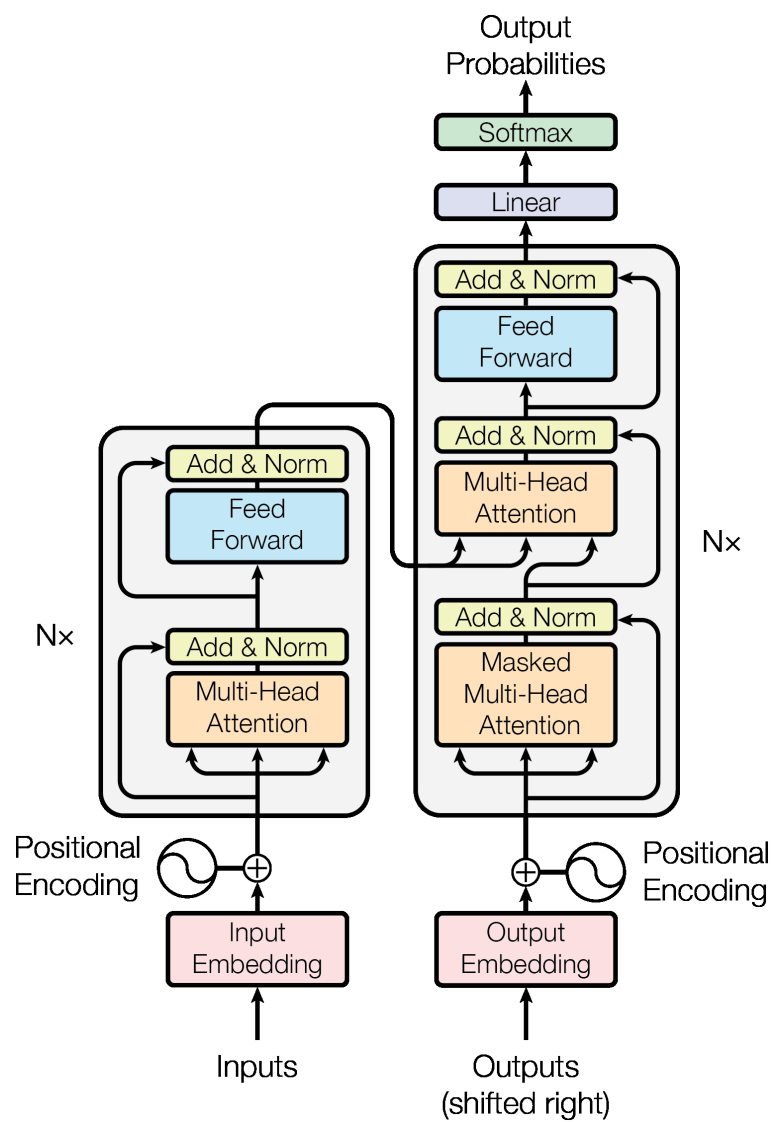


Figura 1: A arquitetura do artigo. Encoder à esquerda, decoder à direita, cada um repetido N vezes. Reproduzida da Figura 1 de Vaswani et al. (2017).

3 Do texto para números

3.1 Quebrar em pedaços

O modelo não trabalha com palavras inteiras, e sim com pedaços de palavra aprendidos estatisticamente, uma técnica chamada **BPE** (*byte-pair encoding*). Antes do treino, o algoritmo percorre o corpus e funde repetidamente o par de símbolos adjacentes mais frequente. Palavras comuns acabam virando um símbolo só; palavras raras permanecem quebradas em pedaços conhecidos (Figura 2).



Figura 2: O BPE segmentando uma palavra rara. Cada linha aplica a fusão de menor *rank*, isto é, a que foi aprendida mais cedo por ser mais frequente. A primeira fusão é o sufixo alemão “-en”.

A vantagem é não ter palavras desconhecidas: qualquer sequência de caracteres pode ser montada a partir de pedaços menores. No nosso modelo, um vocabulário de 9 740 pedaços cobre alemão e inglês juntos, e nas 300 frases de teste não apareceu um único símbolo desconhecido.

Usar um vocabulário **compartilhado** entre as duas línguas não é detalhe: é o que permite amarrar três matrizes que, de outra forma, seriam separadas — o embedding da origem, o do destino e a projeção final antes do softmax passam a ser o mesmo tensor. Isso economiza parâmetros e faz o modelo tratar de forma consistente um pedaço que aparece nas duas línguas.

3.2 Embedding e escala

Cada pedaço vira um vetor de $d_{\text{model}} = 512$ números, buscado numa tabela que é aprendida no treino. Esse vetor é então multiplicado por $\sqrt{d_{\text{model}}} \approx 22,6$.

A razão é a soma que vem logo em seguida, e ela fica clara quando se comparam as duas magnitudes. A codificação posicional tem valores entre -1 e 1 , e como cada par de dimensões contribui com $\sin^2 + \cos^2 = 1$, todo vetor de posição tem norma exatamente $\sqrt{256} = 16$, qualquer que seja a posição.

Os vetores da tabela de embeddings são bem menores que isso. A inicialização sorteia cada peso de uma normal de desvio $d_{\text{model}}^{-1/2}$, o que dá norma esperada $\sqrt{512 \cdot 1/512} = 1,00$, e o treino

praticamente não move esse valor: no modelo treinado a norma média dos 9 739 vetores é 1,04, com 95% deles entre 0,9 e 1,2.

Sem a multiplicação, portanto, o conteúdo entraria na soma valendo $1,04/16 = 6,5\%$ da magnitude da posição. O modelo saberia muito bem *onde* cada pedaço está e quase nada sobre *qual* pedaço é. Com a escala, a norma média passa a 23,5 e a razão se inverte para 1,47 a favor do conteúdo, que é o equilíbrio que se quer: a identidade do pedaço domina, e a posição entra como ajuste (Figura 3).

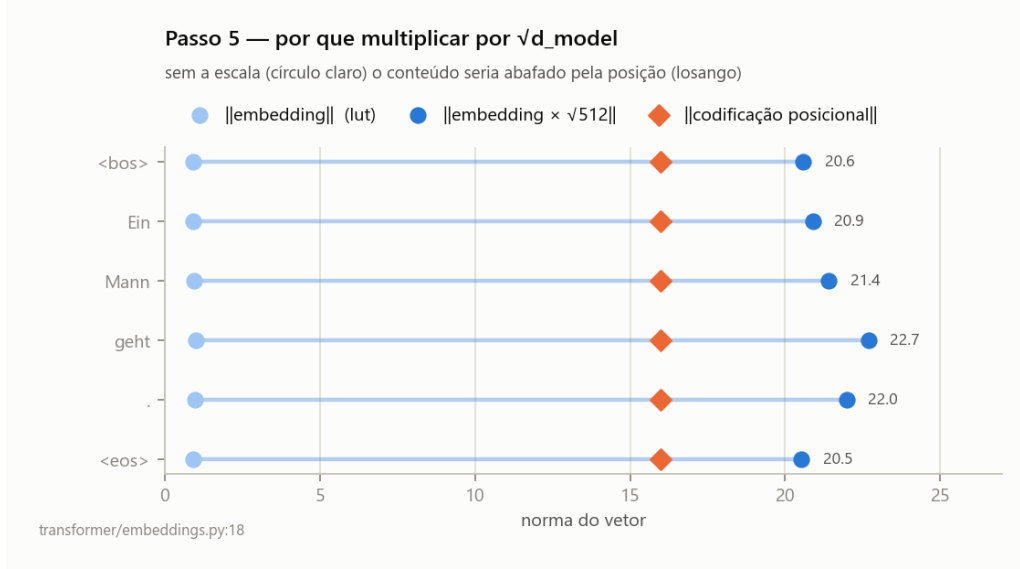


Figura 3: A norma de cada pedaço antes e depois da multiplicação por $\sqrt{d_{\text{model}}}$, com a norma da codificação posicional (losango) como referência. Depois da escala é o conteúdo que domina, e a posição entra como ajuste.

3.3 Onde entra a posição

Como a atenção não distingue ordem, a posição precisa ser injetada explicitamente. O artigo usa uma função fixa, sem parâmetros aprendidos: senos e cossenos de frequências que decaem geometricamente ao longo das dimensões,

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right). \quad (1)$$

O resultado (Figura 4) é que cada posição recebe uma assinatura única: as dimensões baixas oscilam rápido e distinguem posições vizinhas, as altas oscilam devagar e distinguem regiões da frase. Como é uma função e não uma tabela, ela vale para sequências mais longas que as vistas no treino — foi essa a justificativa do artigo para preferi-la a posições aprendidas, que dão resultado praticamente igual mas não extrapolam.

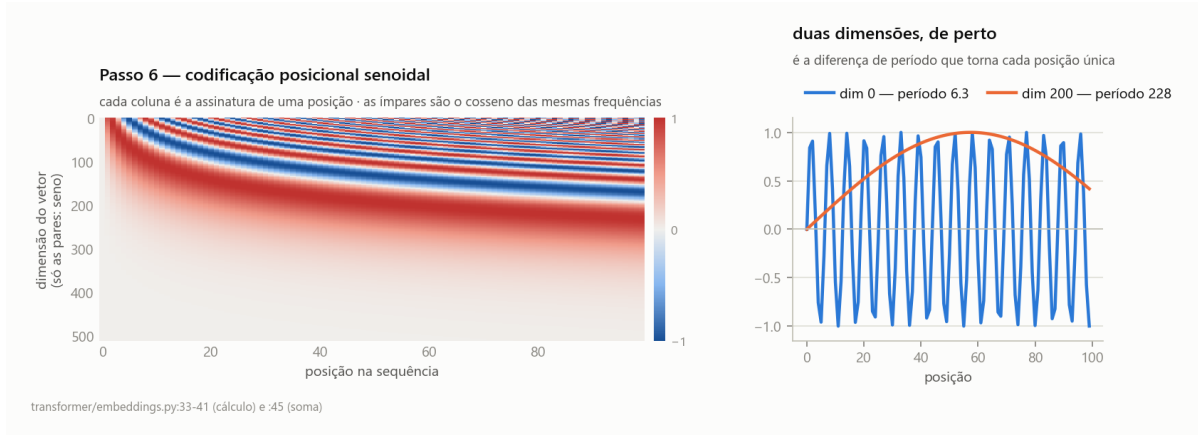


Figura 4: À esquerda, as 100 primeiras posições do vetor posicional. À direita, duas dimensões vistas de perto: a dimensão 0 tem período 6,3 e a 200 tem período 228. É a combinação de escalas que torna cada posição única.

4 Atenção: o mecanismo central

4.1 A ideia

Para cada posição, a atenção responde a uma pergunta: *de quais outras posições eu preciso, e o quanto?* Isso é feito com três projeções lineares do mesmo vetor de entrada, com nomes emprestados de bancos de dados:

- **Query** (Q) — o que esta posição está procurando.
- **Key** (K) — o que cada posição tem a oferecer.
- **Value** (V) — a informação que ela entrega se for escolhida.

O produto $QK^\top$ mede a compatibilidade entre cada query e cada key. Um softmax transforma esses números em pesos que somam 1, e a saída de cada posição é a média dos V ponderada por esses pesos:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (2)$$

É neste ponto, e só neste, que a informação se move entre posições. Todo o resto da arquitetura processa cada posição isoladamente.

4.2 Por que dividir por $\sqrt{d_k}$

A divisão da Equação 2 é o “scaled” do nome e não é cosmética. O produto escalar de dois vetores de dimensão d_k tem variância proporcional a d_k ; com $d_k = 64$, os scores ficam grandes o bastante para saturar o softmax, que passa a dar peso ≈ 1 a uma posição e ≈ 0 a todas as outras. Nessa região o gradiente é praticamente nulo e o modelo para de aprender. A Figura 5 mostra os dois efeitos medidos no modelo treinado.

4.3 Múltiplas cabeças

Em vez de uma atenção sobre vetores de 512 dimensões, o modelo faz $h = 8$ atenções em paralelo sobre subespaços de $d_k = 512/8 = 64$ dimensões, e concatena os resultados (Figura 6). O custo computacional é praticamente o mesmo, e o ganho é que cada cabeça pode se especializar em um tipo de relação diferente.

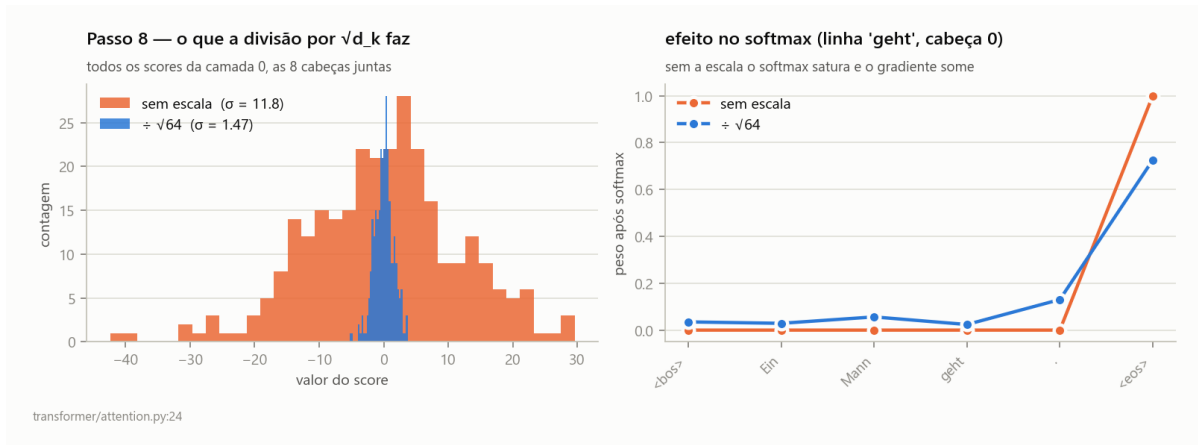
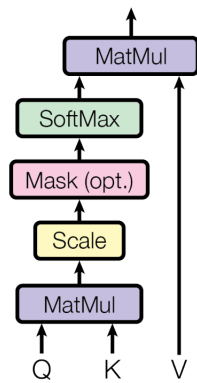


Figura 5: Efeito da divisão por $\sqrt{d_k}$. À esquerda, a distribuição de todos os scores de uma camada: o desvio padrão cai de 11,8 para 1,5. À direita, a consequência no softmax de uma linha: sem a escala ele colapsa em uma única posição.

Scaled Dot-Product Attention



Multi-Head Attention

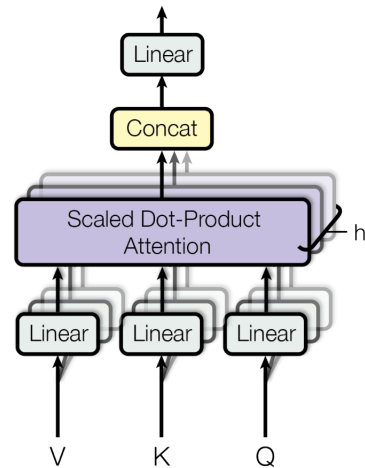


Figura 6: O mecanismo de atenção e sua versão multi-cabeça. Reproduzida da Figura 2 de Vaswani et al. (2017).

Que isso de fato acontece dá para verificar olhando as oito cabeças da mesma camada, na mesma frase (Figura 7): os padrões são visivelmente diferentes entre si.

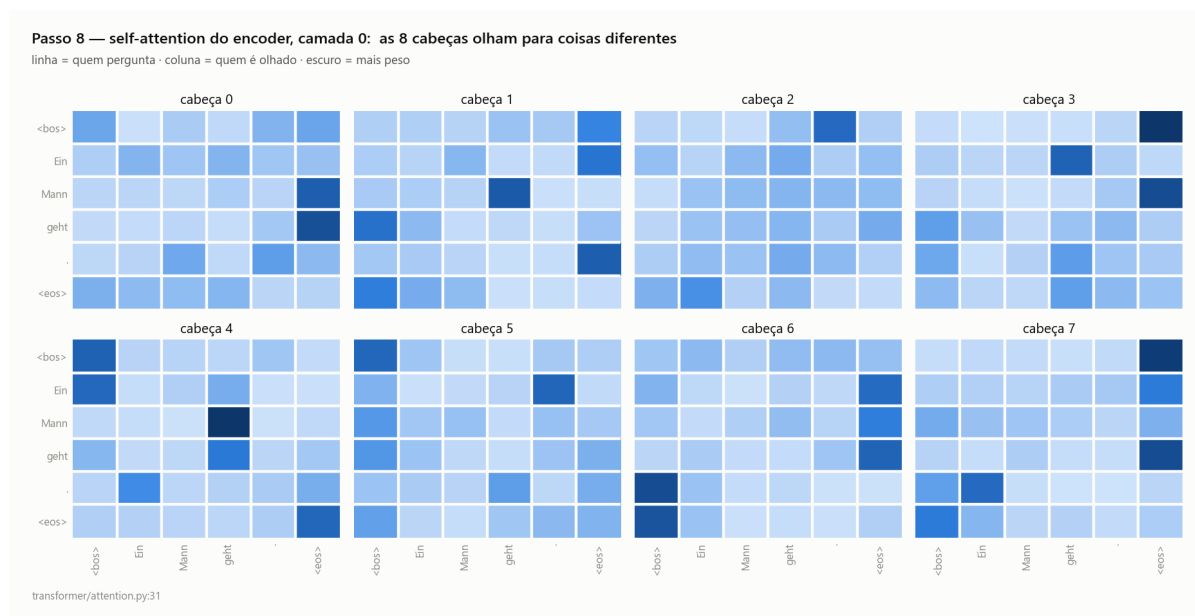


Figura 7: As oito cabeças da primeira camada do encoder, na frase “Ein Mann geht.”. Cada quadro é uma matriz de pesos: a linha é a posição que pergunta, a coluna é a que está sendo olhada.

4.4 Os três usos da atenção

A mesma operação aparece em três lugares, e o que muda é apenas de onde vêm Q , K e V :

Uso	Q , K , V vêm de	Para quê
Self-attention do encoder	todos da frase de origem	relacionar palavras da entrada
Self-attention do decoder	todos do que já foi gerado	manter a saída coerente
Cross-attention	Q do decoder; K , V do encoder	consultar a frase original

Tabela 1: Os três usos da atenção. A cross-attention é o único ponto de contato entre as duas metades do modelo.

5 O bloco que se repete

Em volta de cada sublayer há duas coisas que não mudam nunca:

- Uma **conexão residual**: a entrada é somada à saída, $x + \text{Sublayer}(x)$. Isso dá ao gradiente um caminho direto até as primeiras camadas e é o que torna viável empilhar seis blocos.
- Uma **normalização** (*layer normalization*) aplicada à soma, que devolve média 0 e desvio 1 em cada posição, mantendo os valores numa faixa estável ao longo da pilha.

O outro sublayer, além da atenção, é uma pequena rede densa aplicada *a cada posição separadamente*:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2, \quad (3)$$

que expande de 512 para 2048 dimensões, corta os negativos com ReLU e volta para 512. No modelo treinado, cerca de 93% das ativações intermediárias são zeradas pelo ReLU. A divisão de trabalho é limpa e vale memorizar: **a atenção mistura posições, a rede feed-forward processa cada posição**. Nenhuma outra parte da arquitetura move informação lateralmente.

6 Como a tradução é gerada

6.1 Uma palavra por vez, sem olhar o futuro

O decoder é auto-regressivo: gera um pedaço, acrescenta esse pedaço à sua própria entrada e repete. Durante o treino, porém, queremos prever todas as posições de uma vez, em paralelo — e aí surge um problema. Se o decoder puder ver a frase alvo inteira, prever a próxima palavra é trivial, basta copiá-la.

A solução é uma **máscara causal**: antes do softmax, as posições futuras recebem um valor muito negativo, e portanto peso zero (Figura 8). Cada posição só enxerga a si mesma e as anteriores. Com isso, as previsões de toda a frase podem ser calculadas em um único passe sem que nenhuma delas trapaceie.

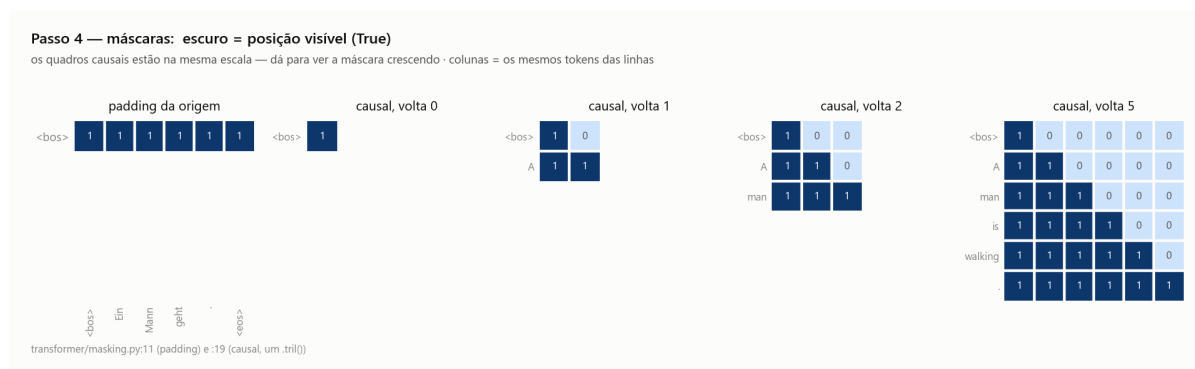


Figura 8: À esquerda, a máscara que marca quais posições da origem são texto de verdade e não preenchimento. À direita, a máscara causal do decoder crescendo a cada pedaço gerado: a posição i só enxerga $j \leq i$.

6.2 Escolhendo o próximo pedaço

A cada volta, a saída do decoder na última posição é projetada para o tamanho do vocabulário e passada por um softmax, virando uma distribuição de probabilidade sobre os 9 740 pedaços possíveis. A Figura 9 mostra as seis voltas que produziram a nossa tradução.

Vale olhar a barra “resto”, que soma tudo o que as cinco primeiras não mostram. Em quatro das seis voltas ela é desprezível — a distribuição está concentrada em meia dúzia de pedaços. Mas na volta 4 ela chega a 22%: ali o modelo divide a aposta entre terminar a frase, continuar com uma preposição e várias outras opções, e é a volta de maior incerteza de todas, mais até que a volta 2. Sem essa barra, um gráfico de top-5 esconde justamente o momento em que o modelo está mais em dúvida.

Escolher sempre a alternativa mais provável (decodificação *greedy*) é simples mas míope: uma escolha ruim no início não pode ser desfeita. É por isso que se usa *beam search*.

6.3 Beam search: quem decide qual frase é a melhor

A resposta curta é que ninguém de fora decide. O juiz é o próprio modelo, e o critério é a probabilidade que ele atribui à frase inteira.

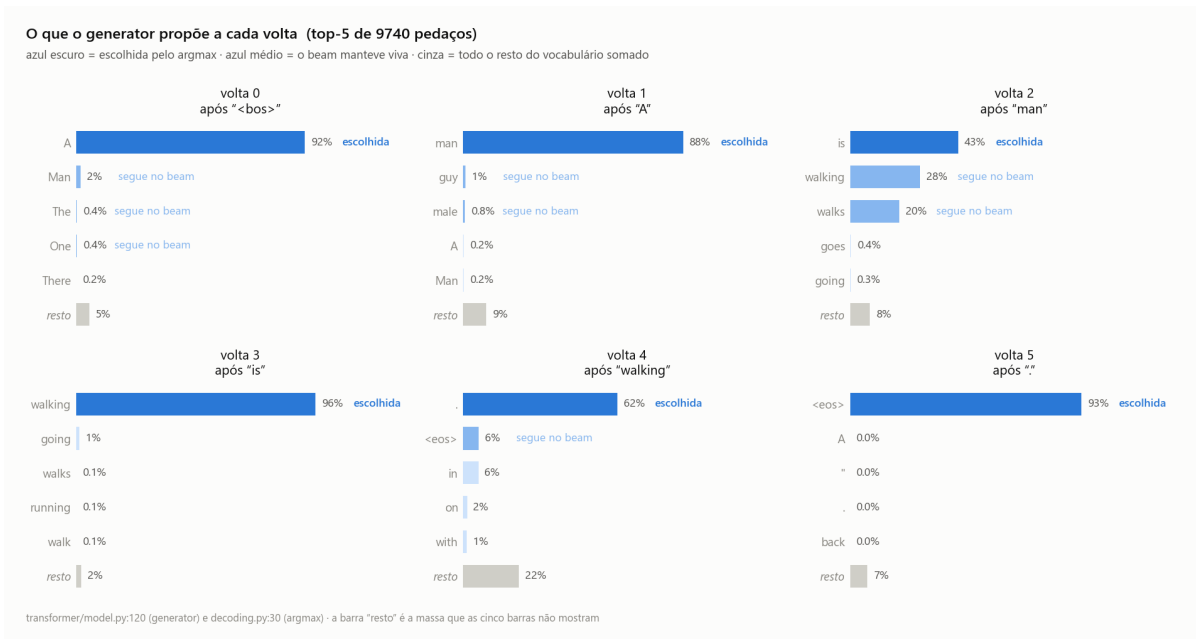


Figura 9: As cinco alternativas mais prováveis em cada volta, mais a massa restante do vocabulário. Azul escuro é a escolhida pelo *argmax*; azul médio marca as que o beam search manteve vivas (Figura 10). Na volta 2 o modelo hesita entre três formas do verbo, e são exatamente essas três que viram feixes irmãos no passo seguinte. Na volta 1 o beam guardou um quarto candidato que não aparece aqui, porque veio de outro prefixo (“Man”) — é o efeito de o *topk* ser global. Na última volta a marcação é omitida: com três hipóteses já encerradas, as vagas restantes são preenchidas com continuações de probabilidade nula.

Como a probabilidade de uma frase é o produto das probabilidades dos pedaços, trabalha-se com o logaritmo, que transforma esse produto numa soma. O **score** de uma hipótese é a soma dos $\log P$ de todos os pedaços que ela gerou até ali. Todo $\log P$ é negativo, então todo score é negativo e *menos negativo é melhor*.

O beam search mantém k hipóteses vivas ao mesmo tempo ($k = 4$ no artigo). A cada passo ele monta a matriz de todas as continuações possíveis — k feixes $\times$ 9 740 pedaços, quase 39 mil candidatos — e escolhe os k melhores **globalmente**. Essa palavra é a chave: não é cada feixe escolhendo o seu melhor pedaço. Um único feixe pode gerar vários dos sobreviventes, e outro pode não gerar nenhum e simplesmente desaparecer.

A Figura 10 abre essa disputa. Cada coluna é um feixe vivo, com o prefixo que ele carrega e o score de onde parte; abaixo dele, as quatro continuações que ele propôs. Em azul, as que passaram no corte; em bege, as que morreram.

Duas coisas ficam visíveis ali. A primeira é o passo 2: **três dos quatro sobreviventes saem de “A man”**, enquanto os feixes “A guy” e “A male”, vivos no passo anterior, propõem continuações perfeitamente razoáveis (*walking*, *is*, *walks*) e não emplacam nenhuma. É a diferença para o greedy, que teria fixado “is” no passo 2 e perdido “walking” e “walks” para sempre.

A segunda é mais sutil e aparece no passo 1. O feixe “The” propôs o pedaço “man” com $\log P = -0,10$ — *melhor* que o $-0,12$ com que o feixe vencedor “A” propôs exatamente o mesmo pedaço. Ainda assim “The man” morreu, porque o que compete não é o $\log P$ do pedaço isolado, e sim o acumulado: $-5,61$ contra $-0,21$. Um começo ruim não se conserta com uma boa continuação — e é justamente por isso que o beam precisa carregar várias hipóteses desde o início, em vez de tentar corrigir o rumo depois.

Quando um feixe gera **<eos>**, a frase está completa: ela é arquivada e sua vaga é liberada, para não continuar competindo. A busca termina quando há k frases completas.

O que cada feixe propôs — e o que o topk matou

azul = passou para o próximo passo · bege = morreu · em cada candidato: logP do pedaço → score acumulado resultante

	feixe 0	feixe 1	feixe 2	feixe 3
	<bos> +0.00	—	—	—
passo 0	A -0.09 → -0.09			
	Man -3.95 → -3.95			
	The -5.51 → -5.51			
	One -5.58 → -5.58			
passo 1	A -0.09	Man -3.95	The -5.51	One -5.58
	man -0.12 → -0.21	walking -0.41 → -4.36	man -0.10 → -5.61	man -0.16 → -5.73
	guy -4.60 → -4.69	walks -1.98 → -5.93	guy -4.92 → -10.43	guy -3.92 → -9.50
	male -4.85 → -4.94	is -2.63 → -6.58	male -5.37 → -10.88	male -4.27 → -9.84
	A -6.00 → -6.08	walk -4.56 → -8.51	A -6.06 → -11.57	A -5.71 → -11.29
passo 2	A man -0.21	Man walking -4.36	A guy -4.69	A male -4.94
	is -0.83 → -1.04	. -0.62 → -4.98	walking -0.77 → -5.46	is -0.68 → -5.62
	walking -1.27 → -1.48	<eos> -1.45 → -5.80	is -1.24 → -5.93	walking -1.34 → -6.27
	walks -1.62 → -1.83	in -3.42 → -7.78	walks -1.82 → -6.51	walks -2.08 → -7.02
	goes -5.44 → -5.65	with -4.11 → -8.47	going -5.26 → -9.95	goes -5.42 → -10.36
passo 3	A man is -1.04	A man walking -1.48	A man walks -1.83	Man walking . -4.98
	walking -0.04 → -1.08	. -0.58 → -2.06	. -0.45 → -2.29	<eos> -0.09 → -5.07
	going -4.31 → -5.35	in -2.36 → -3.84	<eos> -2.26 → -4.09	A -6.40 → -11.38
	walks -6.63 → -7.67	<eos> -2.53 → -4.01	in -3.36 → -5.19	Man -8.05 → -13.03
	running -7.38 → -8.43	with -3.99 → -5.46	with -4.28 → -6.11	" -8.44 → -13.42
passo 4	A man is walking -1.08	A man walking . -2.06	A man walks . -2.29	A man walking in -3.84
	. -0.48 → -1.56	<eos> -0.07 → -2.13	<eos> -0.07 → -2.35	a -0.15 → -3.99
	<eos> -2.76 → -3.84	A -7.89 → -9.95	A -8.77 → -11.05	front -3.57 → -7.41
	in -2.79 → -3.87	" -8.62 → -10.68	" -8.86 → -11.15	the -4.11 → -7.94
	on -4.14 → -5.22	Man -9.15 → -11.21	back -9.40 → -11.68	. -4.64 → -8.47
passo 5	A man is walking . -1.56	(encerrado)	(encerrado)	(encerrado)
	<eos> -0.07 → -1.63			
	A -8.31 → -9.86			
	" -8.76 → -10.32			

transformer/decoding.py:71-76 — os 16 candidatos disputam uma lista só; quem decide é o acumulado, não o logP do pedaço

Figura 10: Todos os candidatos que o decoder avaliou, passo a passo. Cada coluna é um feixe vivo: no cabeçalho, o prefixo que ele carrega e o seu score; abaixo, as quatro continuações mais prováveis que ele propôs, com o $\log P$ do pedaço e o score acumulado que resultaria. Azul = passou no corte dos quatro melhores; bege = morreu ali. São 16 candidatos disputando 4 vagas a cada passo, numa lista única — por isso um feixe pode emplacar três (“A man” no passo 2) e outro nenhum (“The” e “One” no passo 1).

A penalidade de comprimento. Falta uma correção, e é a única coisa aqui que não vem do modelo. Como o score é uma *soma* de números negativos, cada pedaço a mais piora o total: sem intervenção, o modelo preferiria sistematicamente frases curtas e truncadas. A correção de Wu et al., usada no artigo, divide o score pelo comprimento amaciado por um expoente $\alpha = 0,6$:

$$\text{score final} = \frac{\text{score bruto}}{\left(\frac{5+|Y|}{6}\right)^\alpha}. \quad (4)$$

O ranqueamento final das quatro hipóteses da nossa frase:

#	final	bruto	Y	frase
1	-1,075	-1,630	7	A man is walking .
2	-1,482	-2,132	6	A man walking .
3	-1,635	-2,352	6	A man walks .
4	-2,672	-3,844	6	A man is walking

Tabela 2: As quatro frases completas, ordenadas pelo score final. A vencedora é a mais longa das quatro — é a penalidade que torna essa comparação justa.

Isso muda o resultado? Muda com frequência. Nas 60 primeiras frases do conjunto de teste, greedy e beam de 4 produziram traduções **diferentes em 26 delas**, ou 43%. Um exemplo:

DE Ein Mädchen in einem Karateanzug bricht ein Brett mit einem Tritt.
greedy A girl in a karate suit *is wearing* a board with a kick.
beam 4 A girl in a karate suit *takes* a board with a kick.

O greedy se comprometeu com “is” e daí “is wearing” era a continuação natural, que não faz sentido com “board”. O beam manteve o ramo “takes” vivo e ele venceu no acumulado.

Vale ser claro sobre o que o beam *não* faz: ele não avalia gramática, nem sentido, nem fidelidade ao alemão. Ele apenas procura melhor a frase que o modelo já considerava mais provável. Se o modelo estiver errado, o beam encontra o erro com mais eficiência — no exemplo acima, as duas versões ainda traduzem “bricht” (quebra) incorretamente. Beam search melhora a busca, não o modelo.

6.4 O alinhamento aparece sozinho

O resultado mais interessante não é a tradução em si, mas o que a cross-attention revela. Traçando, para cada pedaço gerado, onde o decoder estava olhando na frase alemã, obtém-se um alinhamento entre as duas línguas (Figura 11).

Ninguém forneceu esses alinhamentos. O modelo viu apenas pares de frases, e a correspondência palavra a palavra emergiu como subproduto de aprender a traduzir. É a evidência mais direta de que a atenção está fazendo o que se espera dela.

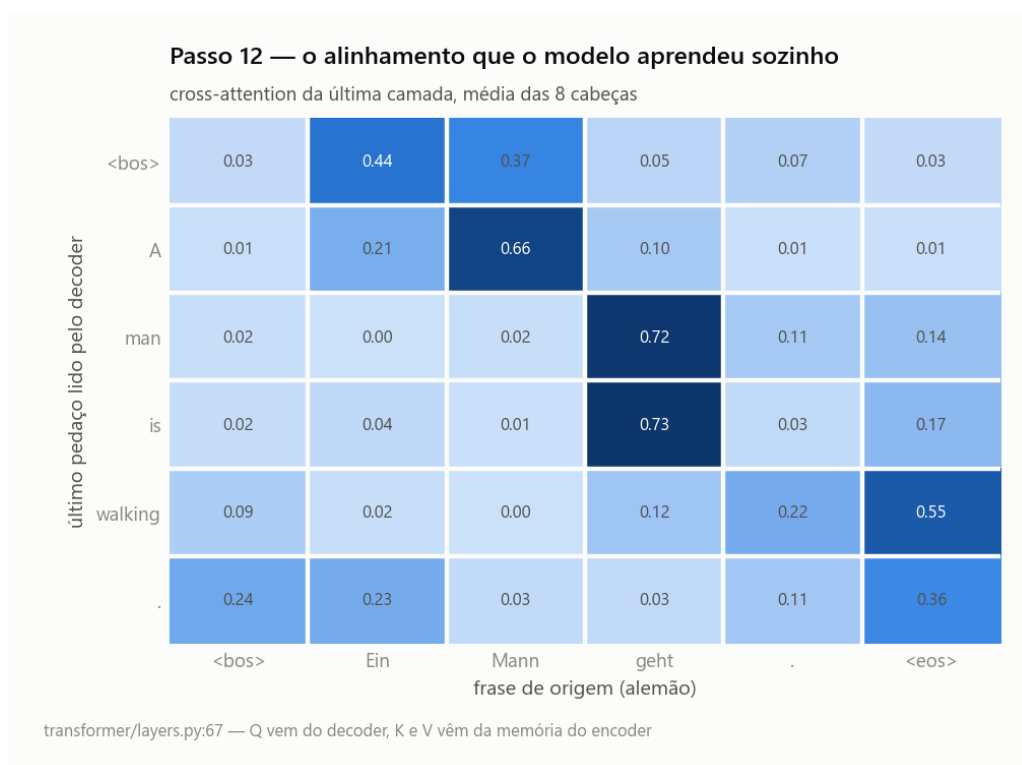


Figura 11: Alinhamento alemão→inglês extraído da cross-attention. Depois de ler “A”, o modelo olha para *Mann*; ao gerar “is” e “walking”, olha para *geht* nas duas voltas — um alinhamento de uma palavra alemã para duas inglesas.

7 Como o modelo aprende

Os capítulos anteriores descrevem o que o modelo calcula. Falta a parte que dá valor aos 49,1 milhões de números que ele carrega. Todos saem de um laço de quatro operações, repetido 8 040 vezes.

7.1 Um passo de treino

Na geração, o decoder produz um pedaço por vez e precisa esperar o anterior ficar pronto. No treino não é assim: a frase-alvo inteira entra de uma vez. A entrada do decoder é a tradução de referência começando em <bos>, e o alvo é a mesma frase deslocada uma casa, terminando em <eos>. A máscara causal (Figura 8) é o que torna isso legítimo: a posição t não enxerga nada à frente dela, então pontuar todas as posições ao mesmo tempo dá o mesmo resultado que pontuar uma por uma. É a razão de o treino ser paralelo e a geração não ser.

Cada batch junta frases de comprimento parecido até somar 2 500 tokens contando o padding, o que deixa 2 183 tokens de alvo reais em média. São 2 183 previsões avaliadas juntas. O conjunto de treino tem 29 000 pares, que cabem em 201 batches, e 40 épocas fecham os 8 040 passos.

Cada passo faz quatro coisas (Figura 12). O forward leva o batch até os logits, uma tabela de 2 183 por 9 740 números. A perda comprime essa tabela inteira em um número só. O backward refaz o caminho ao contrário e devolve, para cada um dos 49,1 milhões de pesos, a derivada da perda em relação àquele peso. O otimizador usa essas derivadas para dar um passo. Depois o batch é descartado e o laço recomeça.

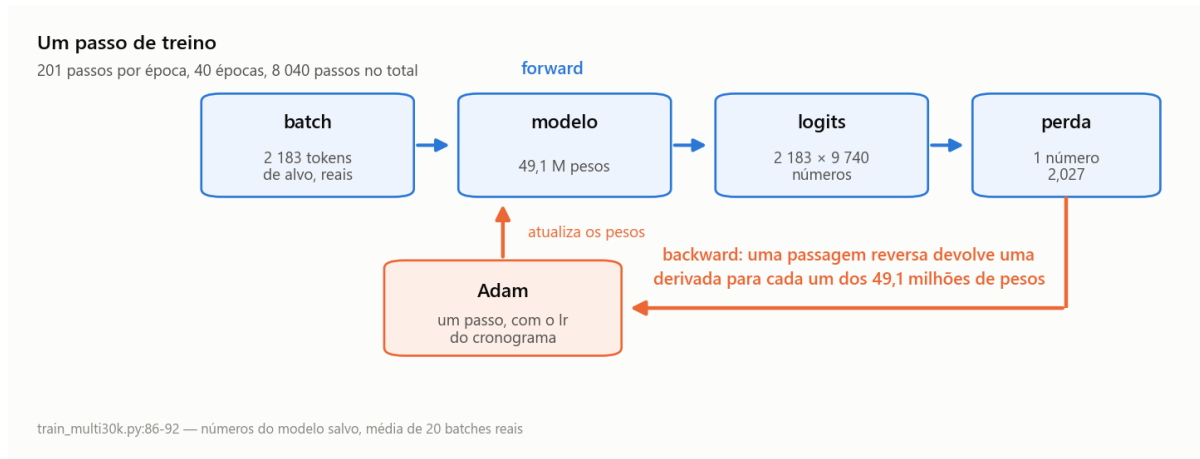


Figura 12: As quatro operações de um passo. O forward atravessa o modelo da esquerda para a direita e termina em um único número; o backward percorre o mesmo caminho ao contrário, uma vez só, e deixa uma derivada em cada peso. Os tamanhos são os do treino deste relatório.

7.2 A perda, e por que ela não chega a zero

O alvo de cada posição não é o pedaço correto com probabilidade 1. Com a suavização de rótulos da seção 5.4 do artigo, 90% da massa fica no pedaço certo e os 10% restantes são espalhados igualmente pelos outros 9 738 do vocabulário. Chame esse vetor de q e a previsão do modelo de p . A perda é a entropia cruzada entre os dois, $-\sum_v q_v \log p_v$, somada nas posições e dividida pelo número delas.

A consequência menos óbvia é que a perda tem um piso acima de zero. Um modelo que acertasse $p = q$ exatamente ainda pagaria

$$-\sum_v q_v \log q_v = -0,9 \log 0,9 - 0,1 \log \frac{0,1}{9\,738} = 1,243. \quad (5)$$

O 2,027 medido no modelo salvo, portanto, não se lê contra zero e sim contra 1,243. O que falta percorrer é 0,78, não 2,03.

O piso é o objetivo, não um efeito colateral. Sem ele, a única forma de zerar a perda seria empurrar o logit do pedaço certo para o infinito, e a capacidade do modelo iria para decorar exemplos em vez de aprender a regularidade. O artigo registra o custo dessa troca: a suavização piora a perplexidade, porque deixa o modelo deliberadamente menos confiante, e ainda assim melhora o BLEU.

7.3 De onde nasce o gradiente

O backward precisa de uma derivada inicial. Ela está na saída e é mais simples do que se imagina: a derivada da perda em relação ao logit do pedaço v é

$$\frac{\partial L}{\partial z_v} = \frac{p_v - q_v}{N}, \quad (6)$$

com N igual ao número de posições do batch. Nenhum softmax, nenhum logaritmo: o previsto menos o alvo. Conferida contra o gradiente que o PyTorch calcula no modelo treinado, a identidade bate com erro máximo de $1,5 \cdot 10^{-6}$.

A Figura 13 mostra as duas situações que a equação (6) produz, em duas posições da mesma frase.

Na posição 2 o modelo tinha que dizer *fixing* e deu a esse pedaço apenas 11,9%. Como p está muito abaixo de q , a derivada fica bem negativa, $-7,8 \cdot 10^{-2}$, e o passo seguinte sobe aquele

logit. Os concorrentes *shoveling*, *walking* e *roof* levam derivadas positivas pequenas e descem um pouco.

Na posição 4 acontece o contrário, e é o caso que explica a suavização melhor do que qualquer descrição dela. O modelo tinha que dizer *roof* e deu 99,0%. Acertou com folga. Mesmo assim a derivada é positiva, $+9,0 \cdot 10^{-3}$, ou seja, o treino *desce* o logit do pedaço correto. O alvo pedia 90% e o modelo entregou 99%, então ele passou do ponto, e a suavização de rótulos é exatamente esse freio.

A intensidade também sai da mesma fórmula. Somando o valor absoluto das derivadas na linha inteira, a posição hesitante movimentava $1,6 \cdot 10^{-1}$ contra $1,9 \cdot 10^{-2}$ da posição confiante, oito vezes mais. O modelo é corrigido onde errou e quase não é tocado onde já estava certo.

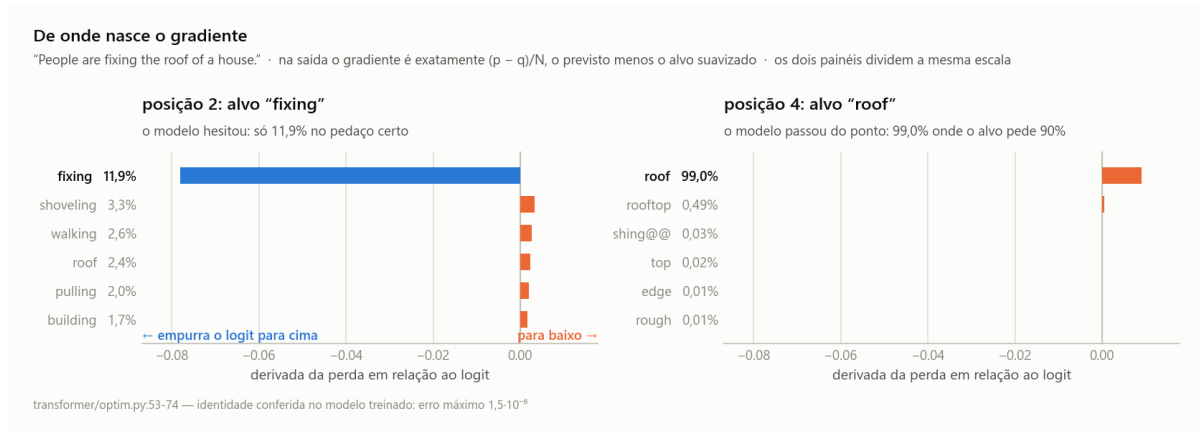


Figura 13: O gradiente na saída do modelo, para os seis pedaços mais prováveis de cada posição. À esquerda o modelo hesitou e leva um empurrão forte para cima no pedaço certo. À direita ele acertou com 99,0% e mesmo assim é empurrado para baixo, porque o alvo suavizado pede 90%. O comprimento da barra é a derivada, e o sinal dela é a direção do ajuste.

7.4 O caminho de volta

Do logit, o gradiente volta pelo generator, atravessa as seis camadas do decoder, entra no encoder pela cross-attention e termina nas tabelas de embedding. O painel esquerdo da Figura 14 mede a norma desse gradiente em cada parada do percurso, no modelo treinado.

O resultado é uma linha quase reta. O gradiente sai dos logits valendo $1,45 \cdot 10^{-4}$ por posição e chega à primeira camada do decoder valendo $1,11 \cdot 10^{-4}$, 77% do que era. Seis camadas de profundidade custam menos de um quarto do sinal.

No encoder o valor é maior, não menor: $1,92 \cdot 10^{-4}$ na camada 1. A explicação é exata. A memória que o encoder produz é consumida pelas seis camadas do decoder, uma vez em cada cross-attention, e o gradiente de um tensor usado em seis lugares é a soma das seis contribuições. Vale reparar no que isso implica: o encoder não tem perda própria. Ele só é corrigido porque o decoder pediu alguma coisa a ele. Quando a cross-attention é uniforme, como no caso do Capítulo 8, o que chega ao encoder é uma média sem informação de posição, e não há como ele aprender alinhamento.

A linha reta não é de graça, e dá para mostrar de onde ela vem. O painel direito repete a medida com uma única alteração no código: a soma residual de cada sublayer, $\text{LN}(x + \text{Sublayer}(x))$, vira $\text{LN}(\text{Sublayer}(x))$. Mesma semente, mesma inicialização, mesmos batches. Com a soma, 87% do gradiente que saiu dos logits chega à camada 1 do decoder. Sem ela chegam 0,0003%, uma queda de trezentas mil vezes em cinco camadas, e o eixo precisa ser logarítmico para o gráfico caber.

A soma residual é o único caminho que atravessa uma camada sem ser multiplicado por matriz nenhuma. Na ida ela preserva o conteúdo, e na volta preserva o gradiente. É o que

torna possível empilhar seis camadas, ou noventa e seis.

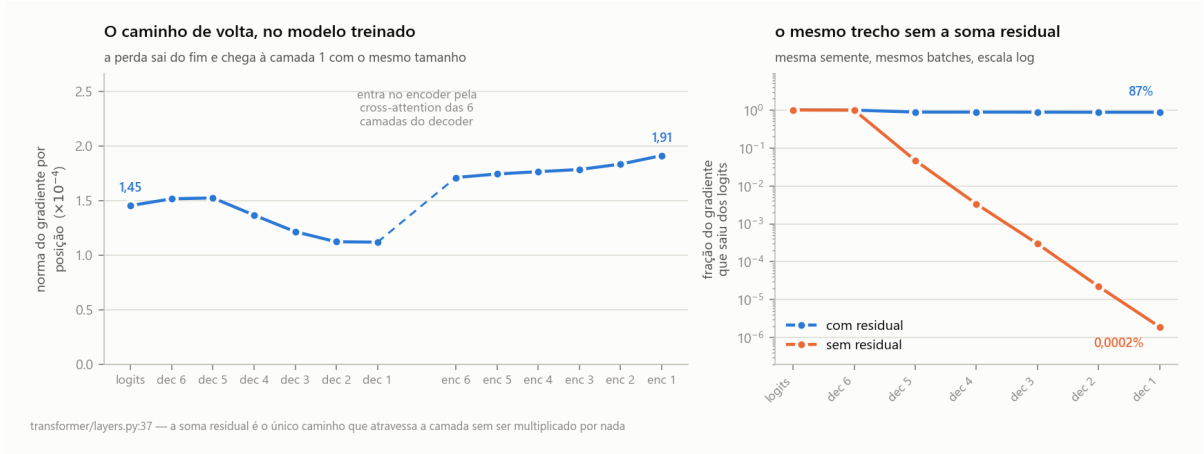


Figura 14: À esquerda, a norma do gradiente em cada parada do caminho de volta no modelo treinado: ela atravessa as doze camadas praticamente inteira. À direita, o mesmo trecho do decoder medido duas vezes na inicialização, mudando só a soma residual. Sem ela o gradiente morre cinco casas decimais antes de chegar à primeira camada.

7.5 O que a passagem reversa precisa guardar

Falta uma conta que o artigo não faz e que decide se um modelo cabe na placa. O backward da Figura 14 não recalcula o forward: ele consome o que o forward deixou guardado. E o que a atenção deixa guardado é a matriz de probabilidades P inteira, de tamanho $L \times L$ por cabeça, porque a derivada do softmax depende dela.

Com as frases curtas do Multi30k isso não incomoda. Num batch de 2500 tokens com $L = 40$, as doze camadas somam alguns megabytes. O problema aparece quando L cresce, porque a conta é quadrática: com $L = 2048$, oito cabeças e um batch de oito frases, a atenção de uma só camada guarda 512 MB de P .

Medindo o forward mais o backward de uma única operação de atenção nesses tamanhos, numa placa de 6 GB, o pico de memória é de 2 152 MB. A mesma conta feita pelo algoritmo do FlashAttention (Dao et al., 2022) usa 313 MB, e o kernel fundido do PyTorch usa 189 MB, um décimo do original. A saída e o gradiente são idênticos nos três, a menos de erro de ponto flutuante.

O truque não é aproximar nada, é não guardar. O forward percorre Q em blocos e mantém o softmax incremental, gravando por linha apenas os *logsumexp*, um vetor de tamanho L . O backward então *recalcula* cada bloco de P a partir de Q , K e desse vetor. Recomputar custa operações de ponto flutuante, mas a atenção é limitada por memória e não por conta, e a troca compensa.

O que faz isso fechar é uma identidade. A derivada do softmax numa posição precisa de $\sum_j p_j \partial L / \partial p_j$, que parece exigir a linha inteira de P de uma vez. Só que essa soma é igual a $dO \cdot O$, o produto entre o gradiente que chegou e a saída que o forward já produziu, ambos de tamanho $L \times d$. Com isso cada bloco precisa de um único número por linha, e a varredura em blocos se sustenta.

A implementação deste repositório traz as duas versões, e o gradiente delas é conferido contra o do PyTorch em precisão dupla.

7.6 O tamanho do passo

O gradiente diz a direção, não a distância. Quem decide a distância é o Adam, que divide cada derivada pela raiz da média quadrática recente daquele mesmo peso. O efeito é padronizar o passo.

Medindo cinco passos consecutivos no pico do cronograma, com o dropout desligado para a medida ser reproduzível, a diferença aparece em duas direções.

Entre blocos de pesos, a norma do gradiente varia por um fator de 2,9: vai de $3,0 \cdot 10^{-4}$ nas FFN do encoder a $8,7 \cdot 10^{-4}$ na tabela de embeddings. A alteração que o Adam de fato aplica varia por um fator de 1,08, entre $3,4 \cdot 10^{-4}$ e $3,7 \cdot 10^{-4}$. A tabela de embeddings recebe quase o triplo do gradiente das FFN do encoder e anda praticamente a mesma distância.

De um batch para o outro a diferença é ainda maior. Ao longo dos cinco passos, a amplitude do gradiente de um mesmo bloco, do menor ao maior valor, vai de 72% a 192% da média daquele bloco, conforme as frases que caíram em cada batch. O passo não acompanha essa oscilação, porque a média quadrática que o Adam guarda vem dos passos anteriores e muda devagar. É o que se quer: o batch decide a direção, não a distância.

É isso que dá sentido ao cronograma da próxima subseção. Como o Adam já removeu a escala do gradiente, a taxa de aprendizado controla de verdade o tamanho do passo, em vez de ser um multiplicador sobre uma quantidade que muda de bloco para bloco.

7.7 Taxa de aprendizado com aquecimento

Em vez de uma taxa fixa, o artigo usa um cronograma que sobe linearmente durante 4 000 passos e depois cai com $1/\sqrt{\text{passo}}$ (Figura 15):

$$lr = d_{\text{model}}^{-0.5} \cdot \min(\text{passo}^{-0.5}, \text{passo} \cdot \text{warmup}^{-1.5}). \quad (7)$$

O aquecimento não é opcional. Como a normalização é aplicada *depois* da soma residual, os gradientes das primeiras camadas são grandes no início do treino, e uma taxa alta desde o passo zero faz o modelo divergir.

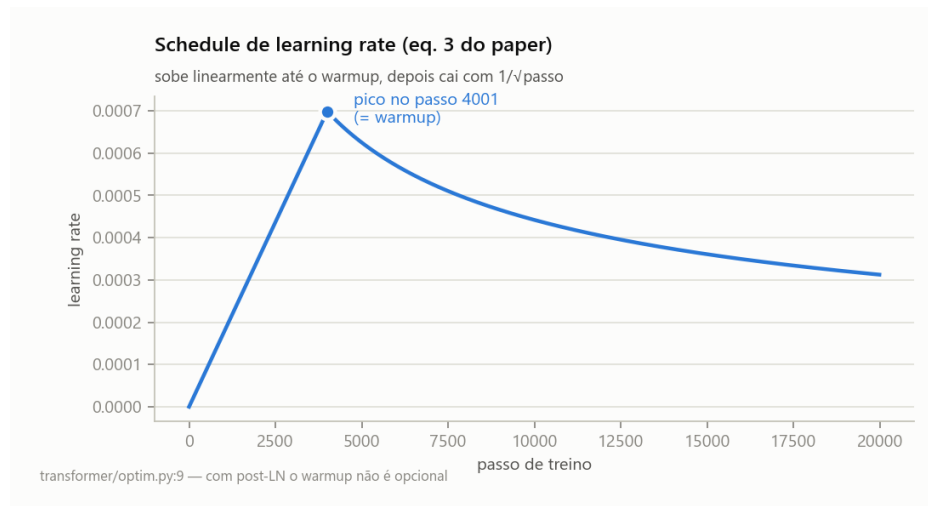


Figura 15: O cronograma de taxa de aprendizado, com o pico exatamente no fim do aquecimento.

7.8 Quando parar

A Figura 16 mostra as 40 épocas. A perda de validação cai até a época 13 e depois sobe, enquanto a de treino continua caindo: daí em diante o modelo está decorando o conjunto de

treino, e as 27 épocas seguintes não melhoram nada.

O que fica salvo não é o checkpoint da época 13 sozinho, e sim a média dos pesos das cinco gravações que terminam nela, como na seção 6.1 do artigo. A média custa uma linha de código e mede-se sozinha: a perda de validação sai de 2,858 no checkpoint isolado para 2,803 na média.

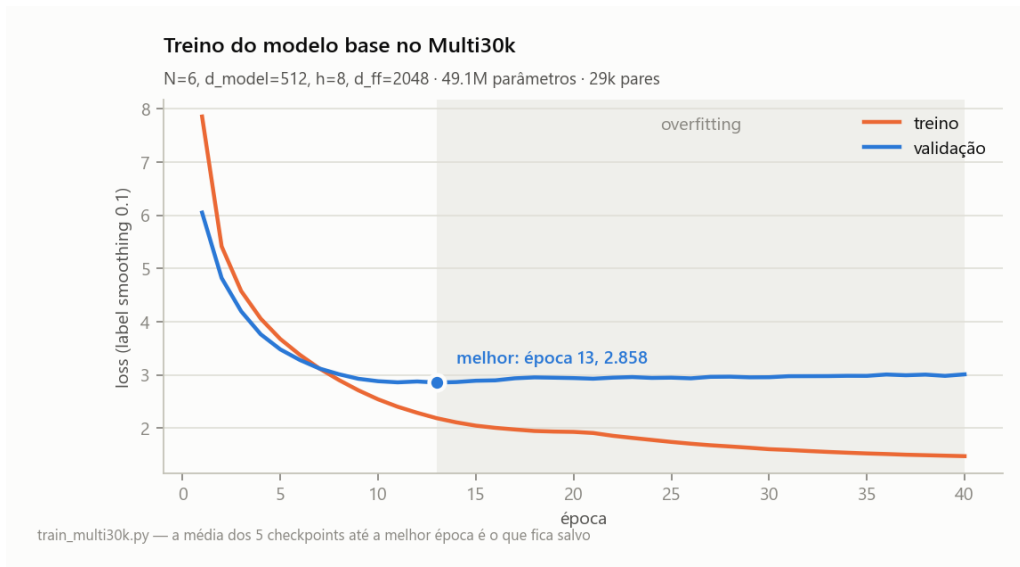


Figura 16: Treino no Multi30k. A partir da época 13 a perda de treino continua caindo enquanto a de validação sobe: o modelo passou a decorar.

8 Uma lição que o artigo não conta

Ao reproduzir a arquitetura fielmente, apareceu um problema que vale registrar, porque ele não está no artigo e é fácil de não perceber.

Cada camada mistura as posições pela atenção, e a normalização em seguida reescala o resultado. Repetindo isso seis vezes com a inicialização usual, as seis posições da saída do encoder já são quase o mesmo vetor *antes de qualquer treino*: a distância média entre elas cai para 18% da distância na entrada. Em um conjunto pequeno como o Multi30k, o treino converge então para um encoder que funciona como um “saco de palavras”, com cross-attention uniforme e sem nenhum alinhamento — e ainda assim traduz razoavelmente, o que torna a falha difícil de notar pela métrica.

A correção é multiplicar por $1/\sqrt{2N}$ as matrizes de saída de cada sublayer na inicialização, o que amortece o ramo residual. Com isso a distância na inicialização passa a 80%, e 96% no modelo treinado (Figura 17). O artigo não especifica a inicialização; esta foi a escolha que fez o modelo aprender alinhamentos em vez de médias.

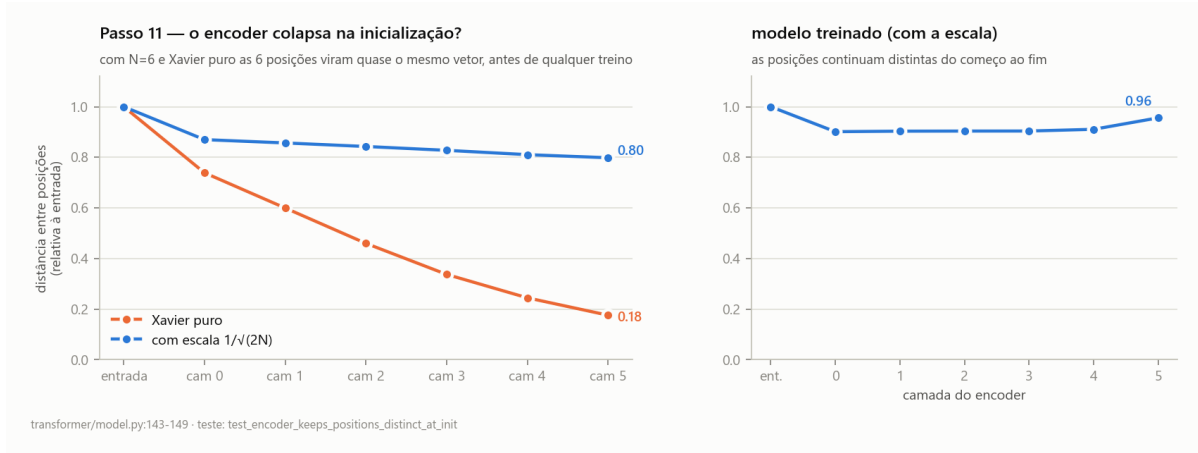


Figura 17: Distância média entre as posições ao longo das camadas do encoder, relativa à entrada. À esquerda, na inicialização, com e sem o amortecimento do ramo residual. À direita, o modelo treinado.

9 Resultados

Configuração	Valor
Camadas N (encoder e decoder)	6
Dimensão do modelo d_{model}	512
Cabeças de atenção h	8
Dimensão por cabeça $d_k = d_v$	64
Dimensão interna da FFN d_{ff}	2048
Dropout	0,1
Vocabulário BPE (compartilhado)	9 740
Parâmetros	49,1 M
Pares de treino (Multi30k)	29 000
Melhor época	13
BLEU-4 em 300 frases de teste	41,15

Tabela 3: Configuração e resultado do modelo treinado. A arquitetura é a *base* do artigo; o conjunto de dados é cerca de 150 vezes menor que o WMT14 usado nele.

Exemplos do conjunto de teste, nenhum visto durante o treino:

Alemão	Referência	Gerado
Leute Reparieren das Dach eines Hauses.	People are fixing the roof of a house.	People are fixing the roof of a house.
Ein Mann geht.	—	A man is walking.
Fünf Leute in Winterjacken und mit Helmen stehen im Schnee.	Five people wearing winter jackets and helmets stand in the snow.	Five people in winter jackets and helmets are standing in the snow.

Tabela 4: Traduções produzidas pelo modelo.

Para contexto: o artigo relata BLEU 27,3 em inglês→alemão no WMT14, uma tarefa muito mais difícil, com 4,5 milhões de pares e frases bem mais longas. Os dois números não são comparáveis; o valor aqui serve apenas para mostrar que a implementação aprende de fato.

10 Em resumo

1. O texto é quebrado em pedaços de palavra, cada pedaço vira um vetor, e a posição é somada a esse vetor.
2. A atenção deixa cada posição olhar diretamente para todas as outras. É a única operação que move informação lateralmente, e o caminho entre duas palavras quaisquer tem comprimento 1.
3. Múltiplas cabeças permitem que relações diferentes sejam capturadas em paralelo, em subespaços distintos.
4. Conexões residuais e normalização são o que tornam possível empilhar seis camadas dessas.
5. O encoder lê a frase uma vez; o decoder gera a tradução pedaço a pedaço, impedido de ver o futuro por uma máscara e consultando o encoder pela cross-attention.
6. O alinhamento entre as línguas não é programado: emerge do treino.

Nada disso é recorrente e nada é convolucional — daí o título do artigo.

Referências

- Vaswani, A. et al. *Attention Is All You Need*. NeurIPS, 2017. <https://arxiv.org/abs/1706.03762>
- Sennrich, R.; Haddow, B.; Birch, A. *Neural Machine Translation of Rare Words with Subword Units*. ACL, 2016. <https://arxiv.org/abs/1508.07909>
- Elliott, D. et al. *Multi30K: Multilingual English-German Image Descriptions*. 2016. <https://arxiv.org/abs/1605.00459>
- Dao, T. et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022. <https://arxiv.org/abs/2205.14135>

As Figuras 1 e 6 são reproduzidas do artigo original; as demais foram geradas a partir do modelo treinado descrito neste relatório. O percurso do mesmo exemplo pelo código, linha a linha, está no README do repositório.